Spaniel

Pseudo-objects and interfaces in Typst

API reference · version 0.0.1

Contents

1 Preface

Please note that this document contains examples that include undocumented functions and variables. These exist only to support the examples and are not part of the public API. They are defined Section 4.1, which describes the scope used when evaluating the examples.

In brief, these terms are as follows: `vector`, `Mul`, `mul_int_int`, `S`, `T`, `U`, `mul_scalar_vector`, `algebra`, `world`, and `mul`. Examples otherwise use the natively documented `Int` type and the `Array[_]` constructor (`array_of`) from the builtin types.

2 Public API

2.1 Types

Type expressions: the vocabulary used to describe runtime types.

Defines nominal types, type constructors, and type variables, along with the structural operations the dispatcher relies on — equality, concreteness checks, variable collection, substitution, keying, and display.

apply_type

Apply a type constructor to type arguments.

```
#display_type(apply_type(Array, Int))
```

```
Array[Int]
```

Parameters

```
apply_type(
  constructor: dictionary ,
  ..arguments: arguments
) -> dictionary
```

constructor `dictionary`

The type constructor to apply.

..arguments `arguments`

Type-expression arguments; the count must equal the constructor's arity.

display_type

Render a type expression as a human-readable string.

```
#display_type(array_of(Int))
```

```
Array[Int]
```

Parameters

```
display_type(ty: dictionary ) -> str
```

ty `dictionary`

The type expression to render.

nominal_type

Construct a nominal runtime type.

`validate` checks the unboxed payload stored in an object of this type.

```
#let Int = nominal_type("demo/Int", name:
"Int", validate: v => type(v) == int)
#Int.name
```

```
Int
```

Parameters

```
nominal_type(
  id: str,
  name: str none,
  validate: function
) -> dictionary
```

id `str`

Stable, globally unique identifier for the type.

name `str` or `none`

Human-readable display name; defaults to `id`.

Default: `none`

validate `function`

Validator for the unboxed payload of objects of this type.

Default: `value => true`

substitute_type

Substitute type variables in `ty` according to `bindings`.

Variables absent from `bindings` are left unchanged.

```
#display_type(substitute_type(array_of(T),
("T": Int)))
```

```
Array[Int]
```

Parameters

```
substitute_type(
  ty: dictionary,
  bindings: dictionary
) -> dictionary
```

ty `dictionary`

The type expression to rewrite.

bindings `dictionary`

Mapping from variable name to replacement type expression.

type_constructor

Construct an n-ary type constructor, such as `Vector[_]`.

Its validator receives `(type_arguments, payload)`.

```
#let Vector = type_constructor("demo/Vector",
1, name: "Vector")
#Vector.arity
```

```
1
```

Parameters

```
type_constructor(
  id: str,
  arity: int,
  name: str none,
  validate: function
) -> dictionary
```

id `str`

Stable, globally unique identifier for the constructor.

arity `int`

Number of type arguments the constructor takes.

name `str` or `none`

Human-readable display name; defaults to `id`.

Default: `none`

validate `function`

Validator receiving `(type_arguments, payload)`.

Default: `(args, value) => true`

type_equal

Test whether two type expressions are structurally equal.

Ignores validators and display names.

```
#repr((type_equal(Int, Int), type_equal(Int,
array_of(Int))))
```

```
(true, false)
```

Parameters

```
type_equal(
  lhs: dictionary,
  rhs: dictionary
) -> bool
```

lhs `dictionary`

First type expression.

rhs `dictionary`

Second type expression.

type_key

A stable key for a type expression or pattern.

This intentionally ignores validators and display names.

```
#type_key(array_of(Int))
```

```
apply(spaniel.types.builtin/Array;nominal(spaniel.types.builtin/Int))
```

Parameters

```
type_key(ty: dictionary) -> str
```

ty `dictionary`

The type expression to key.

type_variable

Construct a type variable for use in implementation patterns.

```
#repr(type_variable("T"))
```

```
(kind: "type.variable", name: "T")
```

Parameters

```
type_variable(name: str) -> dictionary
```

name `str`

The variable's name, e.g. `"T"`.

2.2 Runtime objects

Runtime objects: values boxed together with their runtime type.

Provides the boxing constructor and the predicates that let the dispatcher recognise objects, read their type, and validate their payloads.

is_object

Test whether a value is a protocol object.

```
#repr((is_object(object(Int, 42)),
is_object(42)))
```

```
(true, false)
```

Parameters

```
is_object(value: any ) -> bool
```

value `any`

The value to test.

object

Box an unboxed Typst value with a user-defined runtime type.

```
#object(Int, 42).value
```

```
42
```

Parameters

```
object(
  ty: dictionary ,
  value: any
) -> dictionary
```

ty `dictionary`

The concrete runtime type to attach.

value `any`

The unboxed value to box.

payload_valid

Validate an unboxed value against a concrete runtime type.

```
#repr((payload_valid(Int, 42),
payload_valid(Int, "oops")))
```

```
(true, false)
```

Parameters

```
payload_valid(
  ty: dictionary ,
  value: any
) -> bool
```

ty `dictionary`

The concrete type to validate against.

value `any`

The unboxed value to check.

runtime_type

Return the runtime type of a protocol object.

```
#display_type(runtime_type(object(Int, 42)))
```

```
Int
```

Parameters

```
runtime_type(value: dictionary ) -> dictionary
```

value `dictionary`

The protocol object to inspect.

2.3 Operations & implementations

Operations, requirements, and implementations.

The declarations an author writes to describe a generic operation, the implementations that satisfy it for particular type patterns, and the requirements one implementation may place on others.

implementation

Declare an implementation of an operation.

The body is called as `body(context, ..objects)`.

```
#let mul-ii = implementation(
  Mul, (Int, Int), Int,
  (ctx, lhs, rhs) => object(Int, lhs.value *
rhs.value),
  name: "Int × Int -> Int",
)
#mul-ii.name
```

```
Int × Int -> Int
```

Parameters

```
implementation(
  operation: dictionary ,
  inputs: array ,
  output: dictionary ,
  body: function ,
  constraints: array ,
  name: str none ,
  priority: int
) -> dictionary
```

operation `dictionary`

The operation being implemented.

inputs `array`

Input type patterns, one per argument; may contain type variables.

output `dictionary`

Output type expression produced by the implementation.

body `function`

Implementation body, called as `body(context, ..objects)`.

constraints `array`

Requirements (see `requires()`) that must hold for this implementation to apply.

Default: `()`

name `str` or `none`

Human-readable display name; defaults to the operation's name.

Default: `none`

priority `int`

Tie-breaking priority among equally specific implementations; higher wins.

Default: `0`

operation

Declare an operation: a named, fixed-arity dispatch point that implementations can be registered against.

```
#let Mul = operation("demo/Mul.mul", 2, name:
"mul")
#Mul.arity
```

2

Parameters

```
operation(
  id: str,
  arity: int,
  name: str none
) -> dictionary
```

id `str`

Stable, globally unique identifier for the operation.

arity `int`

Number of object arguments the operation accepts.

name `str` or `none`

Human-readable display name; defaults to `id`.

Default: `none`

requires

Require another operation to be resolvable while checking an implementation.

`inputs` is an array of type expressions. If `output` is supplied, matching the required operation's output against it may introduce new bindings.

```
#repr(requires(Mul, (S, T), output:
U).inputs.map(display_type))
```

```
("S", "T")
```

Parameters

```
requires(
  operation: dictionary,
  inputs: array,
  output: dictionary none
) -> dictionary
```

operation `dictionary`

The operation that must be resolvable.

inputs `array`

Input type expressions for the required call; must match `operation`'s arity.

output `dictionary` or `none`

Optional output type expression to match the required operation's output against.

Default: `none`

2.4 Extensions & worlds

Extensions and world building.

Bundles operations and implementations into extensions, then validates and merges them into an immutable dispatch world — checking arities, resolving references, rejecting duplicates, and ensuring every type variable is bound.

build_world

Combine extensions into a validated immutable dispatch world.

```
#let w = build_world(algebra)
#w.implementations.len()
```

```
2
```

Parameters

```
build_world(..extensions: arguments) -> dictionary
```

..extensions `arguments`

The extensions to combine, each produced by `extension()`.

extension

Bundle operations and implementations into an extension for `build_world()`.

```
#extension(operations: (Mul,),
implementations:
(mul_int_int,)).operations.len()
```

```
1
```

Parameters

```
extension(
  operations: array,
  implementations: array
) -> dictionary
```

operations `array`

Operations declared by this extension.

Default: `()`

implementations `array`

Implementations provided by this extension.

Default: `()`

2.5 Dispatch

Multiple dispatch.

Resolves an operation against a world by selecting the single most-specific implementation for the argument types — recursively satisfying constraints and guarding against cycles — then invokes it and revalidates its result.

dispatch

Invoke an operation using multiple dispatch.

```
#repr(dispatch(world, Mul, integer(2),
vector(Int, (1, 2, 3))).value)
```

```
(2, 4, 6)
```

Parameters

```
dispatch(
  world: dictionary,
  operation: dictionary,
  ..arguments: arguments
) -> dictionary
```

world `dictionary`

The dispatch world to resolve against.

operation `dictionary`

The operation to invoke.

..arguments `arguments`

The object arguments to dispatch on.

dispatcher

Produce a convenient operation-specific function.

```
#let mul = dispatcher(world, Mul)
#mul(integer(6), integer(7)).value
```

```
42
```

Parameters

```
dispatcher(
  world: dictionary,
  operation: dictionary
) -> function
```

world `dictionary`

The dispatch world to bind.

operation `dictionary`

The operation to bind.

try_resolve

Resolve an operation for concrete type descriptors without invoking it.

```
#display_type(try_resolve(world, Mul, (Int,
array_of(Int))).value.output)
```

```
Array[Int]
```

Parameters

```
try_resolve(
  world: dictionary,
  operation: dictionary,
  actual_types: array
) -> dictionary
```

world `dictionary`

The dispatch world to resolve against.

operation `dictionary`

The operation to resolve.

actual_types `array`

Concrete type descriptors for each argument.

2.6 Builtin types

Ready-made runtime types for Typst's built-in value kinds.

A convenience layer over `nominal_type` and `type_constructor`: nominal types mirroring Typst's primitives (`int`, `float`, `str`, `bool`), a couple of refinements (`UInt`, `Number`), the `Array[_]` constructor, and paired constructor functions that box a raw value as the matching object.

array_of

Apply `Array` to an element type, yielding the type expression `Array[T]`.

```
#display_type(array_of(Bool))
```

```
Array[Bool]
```

Parameters

```
array_of(element_type: dictionary) -> dictionary
```

element_type `dictionary`

The element type `T`.

boolean

Box a raw boolean as a `Bool` object.

```
#boolean(false).value
```

```
false
```

Parameters

```
boolean(value: bool) -> dictionary
```

value `bool`

The boolean to box.

floating

Box a raw float as a `Float` object.

```
#floating(3.5).value
```

```
3.5
```

Parameters

`floating(value: float) -> dictionary`

value `float`

The float to box.

integer

Box a raw integer as an `Int` object.

```
#integer(42).value
```

```
42
```

Parameters

`integer(value: int) -> dictionary`

value `int`

The integer to box.

number

Box a raw number (`int` or `float`) as a `Number` object.

```
#number(2).value
```

```
2
```

Parameters

`number(value: int float) -> dictionary`

value `int` or `float`

The number to box.

string

Box a raw string as a `String` object.

```
#string("hi").value
```

```
hi
```

Parameters

`string(value: str) -> dictionary`

value `str`

The string to box.

unsigned_integer

Box a raw non-negative integer as a `UInt` object.

```
#unsigned_integer(7).value
```

```
7
```

Parameters

`unsigned_integer(value: int) -> dictionary`

value `int`

The non-negative integer to box.

Int `dictionary`

The type of Typst integers.

```
#repr((payload_valid(Int, 42),
payload_valid(Int, 4.2)))
```

```
(true, false)
```

UInt `dictionary`

The type of non-negative Typst integers.

```
#repr((payload_valid(UInt, 7),
payload_valid(UInt, -1)))
```

```
(true, false)
```

Float `dictionary`

The type of Typst floating-point numbers.

```
#repr((payload_valid(Float, 3.14),
payload_valid(Float, 3)))
```

```
(true, false)
```

Number `dictionary`

The type of Typst numbers — either an `int` or a `float`.

```
#repr((payload_valid(Number, 3),
payload_valid(Number, 3.14)))
```

```
(true, true)
```

String `dictionary`

The type of Typst strings.

```
#repr((payload_valid(String, "hi"),
payload_valid(String, 5)))
```

```
(true, false)
```

Bool `dictionary`

The type of Typst booleans.

```
#repr((payload_valid(Bool, true),
payload_valid(Bool, 1)))
```

```
(true, false)
```

Array `dictionary`

The `Array[T]` type constructor: an array whose every element is a valid `T`.

The contract is recursive — element validity is delegated to `payload_valid` against the element type argument.

```
#display_type(array_of(String))
```

```
Array[String]
```

3 Internal API

The definitions below are private, underscore-prefixed helpers. They are **not** part of the public API and may change without notice; they are documented here for contributors working on the package internals.

3.1 Results

Internal result type.

Lightweight `ok`/`err` values used to thread fallible computations — type matching, constraint solving, and resolution — without panicking, so that callers can report rich diagnostics instead.

_err

Wrap a message as a failed result.

```
#repr(_err("no matching implementation"))
```

```
(
  kind: "result.err",
  message: "no matching implementation",
)
```

Parameters

`_err(message: str) -> dictionary`

message `str`

The error message.

_is_ok

Test whether a result is successful.

```
#repr((_is_ok(_ok(1)), _is_ok(_err("boom"))))
```

```
(true, false)
```

Parameters

`_is_ok(result: dictionary) -> bool`

result `dictionary`

The result to test.

_ok

Wrap a value as a successful result.

```
#repr(_ok(42))
```

```
(kind: "result.ok", value: 42)
```

Parameters

`_ok(value: any) -> dictionary`

value `any`

The success payload.

3.2 Type internals

_is_concrete_type

Test whether a type expression is fully concrete, i.e. contains no type variables or rigid variables.

```
#repr((_is_concrete_type(array_of(Int)),
_is_concrete_type(array_of(T))))
```

```
(true, false)
```

Parameters

`_is_concrete_type(ty: dictionary) -> bool`

ty `dictionary`

The type expression to test.

_is_type_expression

Test whether a value is a well-formed type expression: a nominal type, type variable, valid constructor application, or internal rigid variable.

```
#repr((_is_type_expression(Int),
_is_type_expression(42)))
```

```
(true, false)
```

Parameters

`_is_type_expression(value: any) -> bool`

value `any`

The value to test.

_type_variables

Collect the names of all type variables occurring in `ty`.

```
#repr(_type_variables(apply_type(Array, T)))
```

```
("T",)
```

Parameters

`_type_variables(ty: dictionary) -> array`

ty `dictionary`

The type expression to scan.

3.3 Pattern matching & specificity

Pattern matching and specificity.

Matches type patterns against concrete types (accumulating type-variable bindings) and decides when one implementation's signature is strictly more specific than another's, which drives overload selection in the dispatcher.

_match_signature

Match a whole signature: each pattern against the corresponding actual type, threading one shared set of bindings across all positions.

```
#let b = _match_signature((S, array_of(T)),
(Int, array_of(Int))).value
#(display_type(b.at("S")) + ", " +
display_type(b.at("T")))
```

```
Int, Int
```

Parameters

```
_match_signature(
  patterns: array,
  actual_types: array
) -> dictionary
```

patterns `array`

The input type patterns of an implementation.

actual_types `array`

The concrete argument types of a call.

_match_type

Match a single type `pattern` against an `actual` type, extending `bindings` with any type variables resolved along the way.

```
#let matched = _match_type(T, Int, (:))
#display_type(matched.value.at("T"))
```

```
Int
```

Parameters

```
_match_type(
  pattern: dictionary,
  actual: dictionary,
  bindings: dictionary
) -> dictionary
```

pattern `dictionary`

The type pattern to match; may contain type variables.

actual `dictionary`

The concrete type to match against.

bindings `dictionary`

Type-variable bindings accumulated so far.

_more_specific

Test whether implementation `lhs` is strictly more specific than `rhs`: `rhs` accepts every call `lhs` does, but not vice versa.

```
#let generic = implementation(Mul, (S, T),
Int, (ctx, l, r) => l, name: "generic")
#let specific = implementation(Mul, (Int,
Int), Int, (ctx, l, r) => l, name:
"specific")
#repr((_more_specific(specific, generic),
_more_specific(generic, specific)))
```

```
(true, false)
```

Parameters

```
_more_specific(
  lhs: dictionary,
  rhs: dictionary
) -> bool
```

lhs `dictionary`

The implementation being tested for greater specificity.

rhs `dictionary`

The implementation to compare against.

_signature_subsumes

Test whether `general` subsumes `specific`: every call described by `specific` is also described by `general`. The variables in `specific` are made rigid (via `prefix`) so only `general` may generalise over them.

```
#repr((
  _signature_subsumes((S,), (Int,), "a"),
  _signature_subsumes((Int,), (S,), "b"),
))
```

```
(true, false)
```

Parameters

```
_signature_subsumes(
  general: array,
  specific: array,
  prefix: str
) -> bool
```

general `array`

The candidate more-general signature (its variables stay flexible).

specific `array`

The candidate more-specific signature (its variables are made rigid).

prefix `str`

A prefix that makes the rigidified variables unique.

_skolemize_type

Replace every type variable in `ty` with a fresh rigid variable, tagged by `prefix`, so it matches only itself. Used to make a signature's variables opaque while testing subsumption.

```
#display_type(_skolemize_type(array_of(T),
"impl"))
```

```
Array[<T>]
```

Parameters

```
_skolemize_type(
  ty: dictionary,
  prefix: str
) -> dictionary
```

ty `dictionary`

The type expression to rigidify.

prefix `str`

A prefix that makes the generated rigid variables unique.

3.4 Registry internals

_constraint_key

Build a stable string key identifying a constraint, used to detect duplicate implementations.

```
#_constraint_key(requires(Mul, (Int, Int)))
```

```
requires(demo/Mul.mul;nominal(spaniel.types.builtin/Int);nominal(spaniel.types.builtin/Int);none)
```

Parameters

```
_constraint_key(constraint: dictionary) -> str
```

constraint `dictionary`

The constraint to key.

_implementation_key

Build a stable string key identifying an implementation by its operation, input and output types, constraints, and priority.

```
#_implementation_key(mul_int_int)
```

```
demo/Mul.mul(nominal(spaniel.types.builtin/Int),nominal(spaniel.types.builtin/Int))->nominal(spaniel.types.builtin/Int) where  @0
```

Parameters

```
_implementation_key(implementation: dictionary) -> str
```

implementation `dictionary`

The implementation to key.

_validate_implementation_variables

Assert that every type variable used by an implementation's constraints and output is bound by its input patterns (or by an earlier constraint output), panicking with a descriptive message otherwise.

```
#{
  _validate_implementation_variables(mul_scalar_ve
  "validated: every type variable is bound"
}
```

```
validated: every type variable is bound
```

Parameters

```
_validate_implementation_variables(implementation: dictionary) -> none
```

implementation `dictionary`

The implementation to check.

3.5 Dispatch internals

_call_key

Build a stable string key for a call site — an operation plus its concrete argument types — used to detect cyclic requirement resolution.

```
#_call_key(Mul, (Int, Int))
```

```
demo/Mul.mul(nominal(spaniel.types.builtin/Int),nominal(spaniel.types.builtin/Int))
```

Parameters

```
_call_key(
  operation: dictionary,
  actual_types: array
) -> str
```

operation `dictionary`

The operation being called.

actual_types `array`

The concrete argument types of the call.

_resolve

Core resolution routine: find the single most-specific implementation of `operation` for `actual_types`, recursively satisfying each implementation's constraints and guarding against cycles via `stack`. Returns an `ok` result carrying the chosen candidate, or an `err` describing why resolution failed.

```
#let resolved = _resolve(world, Mul, (Int,
Int), ())
#resolved.value.implementation.name
```

```
Int × Int -> Int
```

Parameters

```
_resolve(
  world: dictionary,
  operation: dictionary,
  actual_types: array,
  stack: array
) -> dictionary
```

world `dictionary`

The dispatch world to resolve against.

operation `dictionary`

The operation to resolve.

actual_types `array`

The concrete argument types.

stack `array`

Call keys currently being resolved, used for cycle detection.

3.6 Builtin type internals

_namespace `str`

Shared id prefix for the built-in types, keeping their ids unique and namespaced apart from user-defined types.

```
#_namespace
```

```
spaniel.types.builtin
```

4 Appendix

4.1 Example scope

```
#import "/src/lib.typ": *
#import "/src/types/builtin.typ": Int, array_of
#import "/src/lib.typ" as m-lib
#import "/src/internal.typ" as m-internal
#import "/src/types.typ" as m-types
#import "/src/matching.typ" as m-matching
#import "/src/registry.typ" as m-registry
#import "/src/dispatch.typ" as m-dispatch
#import "/src/types/builtin.typ" as m-builtin

// `Int` and the `Array[_]` type (`array_of`) are provided natively by
// `src/types/builtin.typ`, so the examples use those directly. The library has
// no single-call constructor for a boxed array value, so this small helper
// stays a fixture.
#let vector = (element_type, values) => object(array_of(element_type), values)

#let Mul = operation("demo/Mul.mul", 2, name: "mul")

#let mul_int_int = implementation(
  Mul,
  (Int, Int),
  Int,
  (ctx, lhs, rhs) => object(Int, lhs.value * rhs.value),
  name: "Int × Int -> Int",
)

#let S = type_variable("S")
#let T = type_variable("T")
#let U = type_variable("U")

#let mul_scalar_vector = implementation(
  Mul,
  (S, array_of(T)),
  array_of(U),
  (ctx, lhs, rhs) => vector(
    ctx.bindings.at("U"),
    rhs.value.map(value => (
      (ctx.dispatch)(Mul, lhs, object(ctx.bindings.at("T"), value)).value
    )),
  ),
  constraints: (requires(Mul, (S, T), output: U),),
  name: "S × Array[T] -> Array[U]",
)

#let algebra = extension(
  operations: (Mul,),
```

```
  implementations: (mul_int_int, mul_scalar_vector),
)
#let world = build_world(algebra)
#let mul = dispatcher(world, Mul)

#let example-scope = (
  dictionary(m-lib)
    + dictionary(m-internal)
    + dictionary(m-types)
    + dictionary(m-matching)
    + dictionary(m-registry)
    + dictionary(m-dispatch)
    + dictionary(m-builtin)
    + (
      vector: vector,
      Mul: Mul,
      mul_int_int: mul_int_int,
      S: S,
      T: T,
      U: U,
      mul_scalar_vector: mul_scalar_vector,
      algebra: algebra,
      world: world,
      mul: mul,
    )
```